```
from google.colab import drive
drive.mount('/content/drive/')
```

```
Drive already mounted at /content/drive/; to attempt to forcibly remount, call drive.mount("/content/drive/", force_remount=True).
```

```
!pip install spectral
```

```
Looking in indexes: https://pypi.org/simple, https://us-python.pkg.dev/colab-wheels/public/simple/
Requirement already satisfied: spectral in /usr/local/lib/python3.9/dist-packages (0.23.1)
Requirement already satisfied: numpy in /usr/local/lib/python3.9/dist-packages (from spectral) (1.22.4)
```

```
%tensorflow_version 2.x
import tensorflow as tf
import random
from tensorflow.keras import Sequential,layers
from tensorflow.keras.models import Model
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import ModelCheckpoint
import statistics
from sklearn.decomposition import PCA
from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report, cohen_kappa_score
import tensorflow_probability as tfp
from operator import truediv
from tensorflow.compat.v1.distributions import Bernoulli
from plotly.offline import init_notebook_mode

import numpy as np
import matplotlib.pyplot as plt
import scipy.io as sio
import os
import spectral
import tensorflow as tf
from tensorflow.python.keras import backend as K

init_notebook_mode(connected=True)
%matplotlib inline
```

```
Colab only includes TensorFlow 2.x; %tensorflow_version has no effect.
```

```
# Global variables
im_width, im_height, im_depth, im_channel = 11,11,30, 1
mc_loss_weight = 5
```

Model Construction

[ ] ↳ *5 cells hidden*

Data Loading

[ ] ↳ *10 cells hidden*

Training

```
ProtoModel = Prototypical(model,im_width, im_height, im_depth, im_channel)
optimizer = tf.keras.optimizers.Adam(0.001)          #Adam(0.001)
n_times = 15
#std_list = []
```

```
# Metrics to gather
train_loss = tf.metrics.Mean(name='train_loss')
train_acc = tf.metrics.Mean(name='train_accuracy')

def train_step(support, query, support_labels, query_labels, K, C, N):
    # Forward & update gradients
    with tf.GradientTape() as tape:
        loss, mean_accuracy, mean_predictions = ProtoModel(support, query, support_labels, query_labels, K, C, N,n_times,training=True)
    gradients = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(gradients, model.trainable_variables))

    # Log loss and accuracy for step
```

```
    train_loss(loss)
    train_acc(mean_accuracy)
```

```
checkpoint_dir = '/content/drive/My Drive/Research/SAMPLE_CODE_TF2_Keras/Inprogress/Hyperspectral_Classification/Training checkpoints/IP/
checkpoint_prefix = os.path.join(checkpoint_dir, "ckpt")
checkpoint = tf.train.Checkpoint(optimizer=optimizer,
                                 ProtoModel = ProtoModel)
```

```
n_episodes = 100
temp_list = []
for epoch in range(50): # n_epochs-140
    train_loss.reset_states()
    train_acc.reset_states()

    for epi in range(n_episodes):
        tquery_patches, tsupport_patches, query_labels, support_labels = new_episode(patches_class_ip,K1,C,N,train_class_labels)
        train_step(tsupport_patches, tquery_patches,support_labels, query_labels, K1, C, N)
        template = 'Epoch {}, Episode {}, Train Loss: {:.2f}, Train Accuracy: {:.2f}'
        print(template.format(epoch+1, epi+1,train_loss.result(),train_acc.result()*100))
    if epoch % 5 == 0 and epoch != 0 :
        checkpoint.save(file_prefix = checkpoint_prefix)
```

```
Epoch 50, Episode 43, Train Loss: 0.49, Train Accuracy: 100.00
Epoch 50, Episode 44, Train Loss: 0.52, Train Accuracy: 100.00
Epoch 50, Episode 45, Train Loss: 0.51, Train Accuracy: 100.00
Epoch 50, Episode 46, Train Loss: 0.50, Train Accuracy: 100.00
Epoch 50, Episode 47, Train Loss: 0.50, Train Accuracy: 100.00
Epoch 50, Episode 48, Train Loss: 0.49, Train Accuracy: 100.00
Epoch 50, Episode 49, Train Loss: 0.48, Train Accuracy: 100.00
Epoch 50, Episode 50, Train Loss: 0.48, Train Accuracy: 100.00
Epoch 50, Episode 51, Train Loss: 0.50, Train Accuracy: 100.00
Epoch 50, Episode 52, Train Loss: 0.49, Train Accuracy: 100.00
Epoch 50, Episode 53, Train Loss: 0.49, Train Accuracy: 100.00
Epoch 50, Episode 54, Train Loss: 0.48, Train Accuracy: 100.00
Epoch 50, Episode 55, Train Loss: 0.50, Train Accuracy: 100.00
Epoch 50, Episode 56, Train Loss: 0.50, Train Accuracy: 100.00
Epoch 50, Episode 57, Train Loss: 0.49, Train Accuracy: 100.00
Epoch 50, Episode 58, Train Loss: 0.49, Train Accuracy: 100.00
Epoch 50, Episode 59, Train Loss: 0.49, Train Accuracy: 100.00
Epoch 50, Episode 60, Train Loss: 0.50, Train Accuracy: 100.00
Epoch 50, Episode 61, Train Loss: 0.50, Train Accuracy: 100.00
Epoch 50, Episode 62, Train Loss: 0.50, Train Accuracy: 100.00
Epoch 50, Episode 63, Train Loss: 0.49, Train Accuracy: 100.00
Epoch 50, Episode 64, Train Loss: 0.48, Train Accuracy: 100.00
Epoch 50, Episode 65, Train Loss: 0.48, Train Accuracy: 100.00
Epoch 50, Episode 66, Train Loss: 0.48, Train Accuracy: 100.00
Epoch 50, Episode 67, Train Loss: 0.47, Train Accuracy: 100.00
Epoch 50, Episode 68, Train Loss: 0.47, Train Accuracy: 100.00
Epoch 50, Episode 69, Train Loss: 0.47, Train Accuracy: 100.00
Epoch 50, Episode 70, Train Loss: 0.46, Train Accuracy: 100.00
Epoch 50, Episode 71, Train Loss: 0.45, Train Accuracy: 100.00
Epoch 50, Episode 72, Train Loss: 0.45, Train Accuracy: 100.00
Epoch 50, Episode 73, Train Loss: 0.44, Train Accuracy: 100.00
Epoch 50, Episode 74, Train Loss: 0.44, Train Accuracy: 100.00
Epoch 50, Episode 75, Train Loss: 0.43, Train Accuracy: 100.00
Epoch 50, Episode 76, Train Loss: 0.43, Train Accuracy: 100.00
Epoch 50, Episode 77, Train Loss: 0.44, Train Accuracy: 100.00
Epoch 50, Episode 78, Train Loss: 0.43, Train Accuracy: 100.00
Epoch 50, Episode 79, Train Loss: 0.44, Train Accuracy: 100.00
Epoch 50, Episode 80, Train Loss: 0.43, Train Accuracy: 100.00
Epoch 50, Episode 81, Train Loss: 0.43, Train Accuracy: 100.00
Epoch 50, Episode 82, Train Loss: 0.42, Train Accuracy: 100.00
Epoch 50, Episode 83, Train Loss: 0.42, Train Accuracy: 100.00
Epoch 50, Episode 84, Train Loss: 0.42, Train Accuracy: 100.00
Epoch 50, Episode 85, Train Loss: 0.42, Train Accuracy: 100.00
Epoch 50, Episode 86, Train Loss: 0.41, Train Accuracy: 100.00
Epoch 50, Episode 87, Train Loss: 0.41, Train Accuracy: 100.00
Epoch 50, Episode 88, Train Loss: 0.41, Train Accuracy: 100.00
Epoch 50, Episode 89, Train Loss: 0.41, Train Accuracy: 100.00
Epoch 50, Episode 90, Train Loss: 0.40, Train Accuracy: 100.00
Epoch 50, Episode 91, Train Loss: 0.42, Train Accuracy: 100.00
Epoch 50, Episode 92, Train Loss: 0.41, Train Accuracy: 100.00
Epoch 50, Episode 93, Train Loss: 0.42, Train Accuracy: 100.00
Epoch 50, Episode 94, Train Loss: 0.42, Train Accuracy: 100.00
Epoch 50, Episode 95, Train Loss: 0.42, Train Accuracy: 100.00
Epoch 50, Episode 96, Train Loss: 0.42, Train Accuracy: 100.00
Epoch 50, Episode 97, Train Loss: 0.42, Train Accuracy: 100.00
Epoch 50, Episode 98, Train Loss: 0.42, Train Accuracy: 100.00
Epoch 50, Episode 99, Train Loss: 0.41, Train Accuracy: 100.00
Epoch 50, Episode 100, Train Loss: 0.41, Train Accuracy: 100.00
```

Tuning_5

```
tune_set_5 = [[] for i in range(6)]
for j in range(6) :
```

```
  tune_set_5[j] = test_patches_class[j][:5,:,:,:,:]   # for each class first 5 samples taken
std_tune_5 = []
```

```
def tune_episode(tune_set,tC,tK,tN,test_class_labels) :
  selected_classes = np.random.choice(test_class_labels,tC,replace=False)
  support_labels  = list(selected_classes)
  query_labels = []
  support_patches = []
  query_patches = []
  for x in selected_classes :
    y = test_class_labels.index(x)
    np.random.shuffle(tune_set[y])
    support_imgs = tune_set[y][:tK,:,:,:,:]    #Support 1, Query 4
    query_imgs = tune_set[y][tK:5,:,:,:,:]
    support_patches.extend(support_imgs)
    query_patches.extend(query_imgs)
    for i in range(tN) :
      query_labels.append(x)
  temp1 = list(zip(query_patches, query_labels))
  random.shuffle(temp1)
  query_patches, query_labels = zip(*temp1)
  query_patches = tf.convert_to_tensor(np.reshape(np.asarray(query_patches),(tC*tN,im_height,im_width,im_depth,1)),dtype=tf.float32)
  support_patches = tf.convert_to_tensor(np.reshape(np.asarray(support_patches),(tC*tK,im_height,im_width,im_depth,1)),dtype=tf.float32)
  return query_patches, support_patches, query_labels, support_labels
```

```
query_patches, support_patches, query_labels, support_labels = tune_episode(tune_set_5,4,1,4,test_class_labels)
```

```
# Metrics to gather
tune_loss = tf.metrics.Mean(name='tune_loss')
tune_acc = tf.metrics.Mean(name='tune_accuracy')

def tune_step(support, query, support_labels, query_labels, K, C, N):
    # Forward & update gradients
    with tf.GradientTape() as tape:
        loss, mean_accuracy, mean_predictions = ProtoModel(support, query, support_labels, query_labels, K, C, N,n_times,training=True)
    gradients = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(gradients, model.trainable_variables))

    # Log loss and accuracy for step
    tune_loss(loss)
    tune_acc(mean_accuracy)
```

```
checkpoint_dir1 = '/content/drive/My Drive/Research/SAMPLE_CODE_TF2_Keras/Inprogress/Hyperspectral_Classification/Training checkpoints/IP
checkpoint_prefix1 = os.path.join(checkpoint_dir1, "ckpt")
checkpoint = tf.train.Checkpoint(optimizer=optimizer,
                                 ProtoModel = ProtoModel)
```

```
n_episodes = 100
temp_list = []
std_tune_5 = []
for epoch in range(41):
    tune_loss.reset_states()
    tune_acc.reset_states()
    for epi in range(n_episodes+1):
        tquery_patches, tsupport_patches, query_labels, support_labels = tune_episode(tune_set_5,3,1,4,test_class_labels)
        tune_step(tsupport_patches, tquery_patches,support_labels, query_labels, 1, 3, 4)

    template = 'Epoch {}, Tune Loss: {:.2f}, Tune Accuracy: {:.2f}'
    print(template.format(epoch+1,tune_loss.result(),tune_acc.result()*100))
    if (epoch+1)%5 == 0 :
      checkpoint.save(file_prefix = checkpoint_prefix1)
```

```
Epoch 1, Tune Loss: 3.29, Tune Accuracy: 99.83
Epoch 2, Tune Loss: 0.60, Tune Accuracy: 100.00
Epoch 3, Tune Loss: 0.25, Tune Accuracy: 100.00
Epoch 4, Tune Loss: 0.09, Tune Accuracy: 100.00
Epoch 5, Tune Loss: 0.00, Tune Accuracy: 100.00
Epoch 6, Tune Loss: 0.02, Tune Accuracy: 100.00
Epoch 7, Tune Loss: 0.00, Tune Accuracy: 100.00
Epoch 8, Tune Loss: 0.02, Tune Accuracy: 100.00
Epoch 9, Tune Loss: 0.06, Tune Accuracy: 100.00
Epoch 10, Tune Loss: 0.00, Tune Accuracy: 100.00
Epoch 11, Tune Loss: 0.01, Tune Accuracy: 100.00
Epoch 12, Tune Loss: 0.04, Tune Accuracy: 100.00
Epoch 13, Tune Loss: 0.00, Tune Accuracy: 100.00
Epoch 14, Tune Loss: 0.00, Tune Accuracy: 100.00
Epoch 15, Tune Loss: 0.00, Tune Accuracy: 100.00
Epoch 16, Tune Loss: 0.00, Tune Accuracy: 100.00
Epoch 17, Tune Loss: 0.05, Tune Accuracy: 100.00
Epoch 18, Tune Loss: 0.04, Tune Accuracy: 100.00
Epoch 19, Tune Loss: 0.06, Tune Accuracy: 100.00
```

```
Epoch 20, Tune Loss: 0.04, Tune Accuracy: 100.00
Epoch 21, Tune Loss: 0.02, Tune Accuracy: 100.00
Epoch 22, Tune Loss: 0.02, Tune Accuracy: 100.00
Epoch 23, Tune Loss: 0.03, Tune Accuracy: 100.00
Epoch 24, Tune Loss: 0.02, Tune Accuracy: 100.00
Epoch 25, Tune Loss: 0.00, Tune Accuracy: 100.00
Epoch 26, Tune Loss: 0.04, Tune Accuracy: 100.00
Epoch 27, Tune Loss: 0.03, Tune Accuracy: 100.00
Epoch 28, Tune Loss: 0.01, Tune Accuracy: 100.00
Epoch 29, Tune Loss: 0.27, Tune Accuracy: 100.00
Epoch 30, Tune Loss: 1.18, Tune Accuracy: 100.00
Epoch 31, Tune Loss: 0.13, Tune Accuracy: 100.00
Epoch 32, Tune Loss: 0.21, Tune Accuracy: 100.00
Epoch 33, Tune Loss: 0.01, Tune Accuracy: 100.00
Epoch 34, Tune Loss: 0.04, Tune Accuracy: 100.00
Epoch 35, Tune Loss: 0.12, Tune Accuracy: 100.00
Epoch 36, Tune Loss: 0.27, Tune Accuracy: 100.00
Epoch 37, Tune Loss: 0.62, Tune Accuracy: 100.00
Epoch 38, Tune Loss: 0.98, Tune Accuracy: 100.00
Epoch 39, Tune Loss: 7.12, Tune Accuracy: 100.00
Epoch 40, Tune Loss: 20.68, Tune Accuracy: 100.00
Epoch 41, Tune Loss: 32.29, Tune Accuracy: 100.00
```

Testing

```
def test_episode(test_patches_class,test_class_labels,test_C,test_K,i,f) :
  selected_classes = test_class_labels[i:f]   # [1, 2, 3, 4, 5, 6, 7, 8]
  support_labels = list(selected_classes)
  query_labels = []
  support_patches = []
  query_patches = []
  for x in selected_classes :
    y = test_class_labels.index(x)
    support_imgs = test_patches_class[y][:test_K,:,:,:,:]
    query_imgs = test_patches_class[y][test_K:,:,:,:,:]
    support_patches.extend(support_imgs)
    query_patches.extend(query_imgs)
    for i in range(query_imgs.shape[0]) :
      query_labels.append(x)
  temp1 = list(zip(query_patches, query_labels))
  random.shuffle(temp1)
  query_patches, query_labels = zip(*temp1)
  x = len(query_labels)
  query_patches = tf.convert_to_tensor(np.reshape(np.asarray(query_patches),(x,im_height,im_width,im_depth,1)),dtype=tf.float32)
  support_patches = tf.convert_to_tensor(np.reshape(np.asarray(support_patches),(test_C*test_K,im_height,im_width,im_depth,1)),dtype=tf.f
  return query_patches, support_patches, query_labels, support_labels,x
```

```
query_patches, support_patches, query_labels, support_labels,x = test_episode(test_patches_class,test_class_labels,3,5,0,3)
```

```
# Metrics to gather
test_loss = tf.metrics.Mean(name='test_loss')
test_acc = tf.metrics.Mean(name='test_accuracy')
def test_step(support, query, support_labels, query_labels, K, C, y):
    loss, mc_predictions, mean_accuracy, classwise_mean_acc, y = ProtoModel(support, query, support_labels, query_labels, K, C, y,n_times
    return loss, mc_predictions, mean_accuracy, classwise_mean_acc, y
```

```
for epoch in range(1): #1000
    test_loss.reset_states()
    test_acc.reset_states()
    tquery_patches1, tsupport_patches1, query_labels1, support_labels1, x1 = test_episode(test_patches_class,test_class_labels,3,5,0,3)
    loss1, mc_predictions1, mean_accuracy1, classwise_mean_acc1, y1 = test_step(tsupport_patches1, tquery_patches1,support_labels1, query
    print('OA1',mean_accuracy1)
# Class-wise Accuracy
    for i in range(tC) :
      print('class',i+1,classwise_mean_acc1[i])
    print('loss',loss1)
    tquery_patches2, tsupport_patches2, query_labels2, support_labels2, x2 = test_episode(test_patches_class,test_class_labels,3,5,3,6)
    loss2, mc_predictions2, mean_accuracy2, classwise_mean_acc2, y2 = test_step(tsupport_patches2, tquery_patches2,support_labels2, query
    print('OA2',mean_accuracy2)
# Class-wise Accuracy
    for i in range(tC) :
      print('class',i+4,classwise_mean_acc2[i])
    print('loss',loss2)
```

```
OA1 tf.Tensor(0.4391892, shape=(), dtype=float32)
class 1 [1.0]
class 2 [0.28448275862068967]
class 3 [1.0]
loss tf.Tensor(175922.52, shape=(), dtype=float32)
```

```
OA2 tf.Tensor(0.9570957, shape=(), dtype=float32)
class 4 [1.0]
class 5 [0.935]
class 6 [1.0]
loss tf.Tensor(8235.278, shape=(), dtype=float32)
```

Overall Accuracy

```
mean_predictions1 =  tf.reduce_mean(mc_predictions1,axis=0)
mean_predictions2 =  tf.reduce_mean(mc_predictions2,axis=0)
overall_predictions = tf.concat([mean_predictions1,mean_predictions2],axis=0)
overall_true_labels = tf.concat([y1,y2],axis=0)
correct_pred = tf.cast(tf.equal(                                          # accuracy for the current pass
            tf.cast(tf.argmax(overall_predictions, axis=-1), tf.int32),
            tf.cast(tf.argmax(overall_true_labels,axis=-1), tf.int32)), tf.float32)
o_acc = tf.reduce_mean(correct_pred)
print("Overall accuracy:",o_acc)
```

```
Overall accuracy: tf.Tensor(0.7011686, shape=(), dtype=float32)
```

Confusion Matrix

```
from sklearn.metrics import confusion_matrix
from sklearn.metrics import accuracy_score
from sklearn.metrics import classification_report
mean_predictions1 =  tf.reduce_mean(mc_predictions1,axis=0)
cm_pred1 = tf.argmax(mean_predictions1, axis=-1)
mean_predictions2 =  tf.reduce_mean(mc_predictions2,axis=0)
cm_pred2 = tf.argmax(mean_predictions2, axis=-1) + 3
overall_predictions = tf.concat([cm_pred1,cm_pred2],axis=0)
cm_true1 = tf.argmax(y1,axis=-1)
cm_true2 = tf.argmax(y2,axis=-1) + 3
overall_true_labels = tf.concat([cm_true1,cm_true2],axis=0)
results = confusion_matrix(overall_true_labels,overall_predictions)
print ('Confusion Matrix :')
print(results)
print ('Report : ')
print (classification_report(overall_true_labels, overall_predictions))
```

```
Confusion Matrix :
[[ 41   0   0   0   0   0]
 [161  66   5   0   0   0]
 [  0   0  23   0   0   0]
 [  0   0   0  15   0   0]
 [  0   0   0   7 187   6]
 [  0   0   0   0   0  88]]
Report :
              precision    recall  f1-score   support

           0       0.20      1.00      0.34        41
           1       1.00      0.28      0.44       232
           2       0.82      1.00      0.90        23
           3       0.68      1.00      0.81        15
           4       1.00      0.94      0.97       200
           5       0.94      1.00      0.97        88

    accuracy                           0.70       599
   macro avg       0.77      0.87      0.74       599
weighted avg       0.92      0.70      0.71       599
```

Kappa Accuracy

```
import sklearn
sklearn.metrics.cohen_kappa_score(overall_true_labels, overall_predictions)
```

```
0.6284286511344221
```

0s completed at 2:52 PM